Programming with Python & Problem Solving (05MC0107)
Lab Manual (2026-27)

Program: MCA

Semester: 1

Name of the Student: HEET POPAT

Enrolment No.:92600584084

INDEX

Lab	Program	Date	Remarks	Signature
1	Write a program to demonstrate basic input and output operations in python.			
2	Write a program to illustrate the use of different data types and type casting.			
3	Write a program to perform arithmetic, relational and logical operations using Python operators.			
4	Write a program to demonstrate string operations including slicing, formatting and built-in string functions.			
5	Write a program to create and manipulate lists using indexing, slicing and list comprehensions.			
6	Write a program to illustrate the use of tuples and sets with basic operations.			
7	Write a program to create a dictionary and demonstrate dictionary methods and iteration.			
8	Write a program to explain mutable and immutable objects in Python.			
9	Write a program to define and use user-defined functions with different types of arguments.			
10	Write a program to demonstrate recursion using factorial or Fibonacci series.			
11				
12				
13				
14				
15				
16				
17				
18				
19				
20				
21				
22				
23				
24				
25				

26				
27				
28				
29				
30				
31				
32				
33				
34				
35				
36				

Q1

Definition:

Write a program to demonstrate basic input and output operations in Python.

Aim/Objective:

To write and execute a Python program that demonstrates basic input and output operations using `input()` and `print()` functions.

Source Code:

```
"""Program 01 :- Write a program to demonstrate basic input and
output operations in python."""

name = input("Enter your name: ")
age = input("Enter your age: ")

print("\nHello,", name)
print("Your age is", age)
```

```
Enter your name: Heet
Enter your age: 22

Hello, Heet
Your age is 22
PS C:\Users\popat\Desktop\Python-Programm
-Assignments>
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate basic input and output operations. The `input()` function was used to accept data from the user, while the `print()` function was used to display the output.

Q2

Definition:

Write a program to illustrate the use of different data types and type casting.

Aim/Objective:

To write and execute a Python program that demonstrates the use of different data types and type casting.

Source Code:

```
"""Program 02:- Write a Program to demonstrate different data
types and type casting """
integer_value = 25
float_value = 10.5
string_value = "50"
```

```
boolean_value = True


print("Integer value: ",integer_value)
print("Float Value: ", float_value)
print("String Value: ", string_value)
print("Boolean Value: ",boolean_value)

converted_integer = int(string_value)
converted_float = float(integer_value)
converted_string = str(integer_value)

print("\nAfter Type Casting: ")
print("String to Integer:", converted_integer)
print("Integer to Float:",converted_float)
print("Integer to String:",converted_string)
```

```
Integer value:  25
Float Value:  10.5
String Value:  50
Boolean Value:  True

After Type Casting:
String to Integer: 50
Integer to Float: 25.0
Integer to String: 25
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate different data types and type casting. The program showed how values can be stored using different data types and converted from one type to another using functions such as `int()`, `float()`, and `str()`.

Q3

Definition:

Write a program to perform arithmetic, relational and logical operations using Python operators.

Aim/Objective:

To write and execute a Python program to demonstrate arithmetic, relational, and logical operations using Python operators.

Source Code:

```
""" Write a program to perform arithmetic relational and logical
operations using python operators. """

a = 10
b = 5
```

```python
print("Arithmetic Operations:")
print("Addition:",a+b)
print("Subtraction:",a-b)
print("Multiplication:",a*b)
print("Division:",a/b)
print("Modulus:",a%b)
print("Exponentiation:",a**b)
print("Floor Division:",a//b)

print("\n Relational Operations:")
print("a == b", a == b)
print("a != b", a != b)
print("a > b:", a > b)
print("a < b:", a < b)
print("a >= b:", a >= b)
print("a <= b:", a <= b)

x = True
y = False

print("\n Logical Operations:")
print("x and y:", a and y)
print("x or y:", x or y)
print("not x:", not x)
```



```
Arithmetic Operations:
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
Modulus: 0
Exponentiation: 100000
Floor Division: 2

 Relational Operations:
a == b False
a != b True
a > b: True
a < b: False
a >= b: True
a <= b: False

 Logical Operations:
x and y: False
x or y: True
not x: False
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate arithmetic, relational, and logical operations. Arithmetic operators were used for mathematical calculations, relational operators were used to compare values, and logical operators were used to combine or evaluate conditions.

Q4

Definition:

Write a program to demonstrate string operations including slicing, formatting and built-in string functions.

Aim/Objective:

To write and execute a Python program to demonstrate string slicing, string formatting, and built-in string functions in Python.

Source Code:

```
""" Write a program to demonstrate string operations including
slicing formatting and build-in string functions. """

text = "Python Programming"

#string slicing
print("Original String:",text)
print("First 6 Characters:",text[:6])
print("Characters from index 7:",text[7:])
print("Characters from index 0 to 5:",text[0:6])

#string formatting
name = "Heet"
language = "Python"
```

```
age = 21
city = "Rajkot"

print("\nString Formatting:")
print("My name is {} and I am learning {}.".format(name,
language))
print("\n I am Learning {} because ahead i want to contribute in
ai ml project in gsoc".format(language))
print("\n I am {} years old and i live in {}".format(age,city))

#Build-in string functions
print("\nBuild-in String Functions:")
print("Length:",len(text))
print("Uppercase:",text.upper())
print("Lowercase:",text.lower())
print("Replace:",text.replace("Python","Java"))
print("Find 'Programming':", text.find("Programming"))
print("Count of 'm':",text.count("m"))
print("count of 'p':",text.count("p"))
```

```
Original String: Python Programming
First 6 Characters: Python
Characters from index 7: Programming
Characters from index 0 to 5: Python

String Formatting:
My name is Heet and I am learning Python.

 I am Learning Python because ahead i want to contribute in ai ml project in gsoc

 I am 21 years old and i live in Rajkot

Build-in String Functions:
Length: 18
Uppercase: PYTHON PROGRAMMING
Lowercase: python programming
Replace: Java Programming
Find 'Programming': 7
Count of 'm': 2
count of 'p': 0
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate string slicing, string formatting, and built-in string functions. These operations help in extracting, formatting, modifying, and analyzing text efficiently in Python.

Q5

Definition:

Write a program to create and manipulate lists using indexing, slicing, and list comprehensions.

Aim/Objective:

To write and execute a Python program to create and manipulate lists using indexing, slicing, and list comprehensions.

Source Code:

```
""" Write a program to create and manipulate lists using
indexing slicing and list comprehensions. """

numbers = [10,20,30,40,50]

print("Original List: ",numbers)

#inexing
print("\nIndexing:")
print("First element:",numbers[0])
print("Third element:",numbers[2])
print("Last element:",numbers[-1])

#slicing
print("\nSlicing:")
print("First three elements:",numbers[:3])
print("Elements from index 2: ",numbers[2:])
print("Elements from index 1 to 3:",numbers[1:4])
```

```python
#Manipulating the list
numbers.append(60)
numbers.remove(20)

print("\nAfter Manipulation:")
print("List:",numbers)

squares = [i ** 2 for i in numbers]

print("\nList Comprehension:")
print("Squares:",squares)
```

```
Original List:  [10, 20, 30, 40, 50]

Indexing:
First element: 10
Third element: 30
Last element: 50

Slicing:
First three elements: [10, 20, 30]
Elements from index 2:  [30, 40, 50]
Elements from index 1 to 3: [20, 30, 40]

After Manipulation:
List: [10, 30, 40, 50, 60]

List Comprehension:
Squares: [100, 900, 1600, 2500, 3600]
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate the creation and manipulation of lists using indexing, slicing, and list comprehension. Indexing was used to access individual elements, slicing was used to access a range of elements, and list comprehension was used to efficiently create a new list.

Q6

Definition:

Write a program to illustrate the use of tuples and sets with basic operations.

Aim/Objective:

To write and execute a Python program to demonstrate the use of tuples and sets with basic operations.

Source Code:

```
""" To write and execute a Python program to demonstrate the use
of tuples and sets with basic operations. """

numbers = (10,20,30,40,50)

print("Tuple:",numbers)

print("\n Tuple Operations:")
print("First element:",numbers[0])
print("Last element:",numbers[-1])
print("Length of tuple:",len(numbers))

print("Tuple slicing:",numbers[1:4])

set1 = {10,20,30,40}
set2 = {30,40,50,60}
```

```
print("\nSet Operations:")
print("Set 1:",set1)
print("Set 2:",set2)

set1.add(70)
print("After adding 70:",set1)

set1.remove(20)
print("After removing 20:",set1)

print("Union:",set1.union(set2))

print("Intersection:",set1.intersection(set2))

print("Difference:",set1.difference(set2))
```

```
Tuple: (10, 20, 30, 40, 50)

 Tuple Operations:
First element: 10
Last element: 50
Length of tuple: 5
Tuple slicing: (20, 30, 40)

Set Operations:
Set 1: {40, 10, 20, 30}
Set 2: {40, 50, 60, 30}
After adding 70: {70, 40, 10, 20, 30}
After removing 20: {70, 40, 10, 30}
Union: {70, 40, 10, 50, 60, 30}
Intersection: {40, 30}
Difference: {10, 70}
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate tuples and sets with basic operations. Tuple indexing, slicing, and length operations were performed, while set operations such as adding, removing, union, intersection, and difference were demonstrated.

Q7

Definition:

Write a program to create a dictionary and demonstrate dictionary methods and iteration.

Aim/Objective:

To write and execute a Python program to create a dictionary and demonstrate various dictionary methods and iteration.

Source Code:

```
""" Program to demonstrate dictionary methods and iteration """

student = {
    "name": "Heet",
    "age": 22,
    "course": "MCA"
}

print("Original Dictionary:", student)


print("\nName:", student["name"])


student["city"] = "Ahmedabad"


student["age"] = 23
```

```
print("\nAfter Adding and Updating:")
print(student)

# Dictionary methods
print("\nDictionary Methods:")
print("Keys:", student.keys())
print("Values:", student.values())
print("Items:", student.items())


student.pop("city")

print("\nAfter Removing City:")
print(student)


print("\nDictionary Iteration:")
for key, value in student.items():
    print(key, ":", value)
```

```
Original Dictionary: {'name': 'Heet', 'age': 22, 'course': 'MCA'}

Name: Heet

After Adding and Updating:
{'name': 'Heet', 'age': 23, 'course': 'MCA', 'city': 'Ahmedabad'}

Dictionary Methods:
Keys: dict_keys(['name', 'age', 'course', 'city'])
Values: dict_values(['Heet', 23, 'MCA', 'Ahmedabad'])
Items: dict_items([('name', 'Heet'), ('age', 23), ('course', 'MCA'), ('city', 'Ahmedabad')])

After Removing City:
{'name': 'Heet', 'age': 23, 'course': 'MCA'}

Dictionary Iteration:
name : Heet
age : 23
course : MCA
PS C:\Users\popat\Desktop\Python-Programming-Assignments>
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate the creation of a dictionary, various dictionary methods, and iteration through key-value pairs. Dictionaries are useful for storing and accessing data using unique keys.

Q8

Definition:

Write a program to explain mutable and immutable objects in Python.

Aim/Objective:

To write and execute a Python program to demonstrate the difference between mutable and immutable objects in Python.

Source Code:

```
""" Program to demonstrate mutable and immutable objects """

numbers = [10, 20, 30]

print("Original List:", numbers)

numbers[0] = 100

print("Modified List:", numbers)

number = 10

print("\nOriginal Integer:", number)

number = number + 5

print("Modified Integer:", number)
```

```python
name = "Python"

print("\nOriginal String:", name)

name = name + " Programming"

print("Modified String:", name)
```

```
Original List: [10, 20, 30]
Modified List: [100, 20, 30]

Original Integer: 10
Modified Integer: 15

Original String: Python
Modified String: Python Programming
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate mutable and immutable objects. The list was modified directly because lists are mutable, whereas integer and string values cannot be changed directly because they are immutable.

Q9

Definition:

Write a program to define and use user-defined functions with different types of arguments.

Aim/Objective:

To write and execute a Python program to demonstrate user-defined functions using different types of arguments such as positional, default, keyword, and variable-length arguments.

Source Code:

```
""" Program to demonstrate user-defined functionswith different
types of arguments """

def add(a, b):
    return a + b

print("Addition:", add(10, 20))


def greet(name="Student"):
    print("Hello,", name)

greet()
greet("Heet")


def student_info(name, age):
    print("Name:", name)
    print("Age:", age)

student_info(age=22, name="Heet")
```

```python
def total(*numbers):
    return sum(numbers)

print("Total:", total(10, 20, 30, 40))
```

```
Addition: 30
Hello, Student
Hello, Heet
Name: Heet
Age: 22
Total: 100
```

Conclusion:

Thus, the Python program was successfully executed to demonstrate user-defined functions with different types of arguments. Positional arguments, default arguments, keyword arguments, and variable-length arguments were used to make functions flexible and reusable.

Q10

Q10(a) Factorial Using Recursion

Definition:

Write a program to demonstrate recursion using factorial.

Aim/Objective:

To write and execute a Python program to calculate the factorial of a number using recursion.

Source Code:

```
""" Program to demonstrate recursion using factorial """

def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

number = 5

result = factorial(number)

print("Number:", number)
print("Factorial:", result)
```

```
Number: 5
Factorial: 120
```

Conclusion:

Thus, the Python program was successfully executed to calculate the factorial of a number using recursion. The function repeatedly calls itself with a smaller value until it reaches the base condition of 0 or 1, after which the results are returned.

Q10(b) Fibonacci Series Using Recursion

Definition:

Write a program to demonstrate recursion using the Fibonacci series.

Aim/Objective:

To write and execute a Python program to generate the Fibonacci series using recursion.

Source Code:

```
""" Program to demonstrate recursion using Fibonacci series """

def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)


terms = 7

print("Fibonacci Series:")

for i in range(terms):
    print(fibonacci(i), end=" ")
```

```
Fibonacci Series:
0 1 1 2 3 5 8
```

Conclusion:

Thus, the Python program was successfully executed to generate the Fibonacci series using recursion. The function calls itself with the previous two positions, and the base condition returns the first two Fibonacci values, 0 and 1.